University of Puerto Rico
Mayagüez Campus
Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Programming Assignment 3 - Solving problems by Search: Search

by
Diego H. Merchan Rueda
Marco A. Monserrat Montoto
Edgard A. Díaz Bobé
Malik J. Paramo Rios

For: Professor Jose F. Vega Riveros
Course: ICOM5015, Section 001D
Date: April 22, 2026

Introduction

Purpose of Assignment

Search is a fundamental topic in Artificial Intelligence because many problems can be represented as states, actions, and transitions that move from an initial state to a goal state. In AI, search refers to the process of exploring possible states and actions in order to find a path that solves a given problem. In this assignment, that idea is applied to two classical problems: finding a shortest path in a plane with polygonal obstacles and solving the missionaries and cannibals problem. For Problem 3.7, the goal is to model a shortest-path problem with obstacles and solve it using an appropriate search method. For Problem 3.9, the goal is to represent the missionaries and cannibals problem, implement a search solution, and identify the optimal sequence of actions. Although these two problems are different in nature, both show that the way a problem is represented has a major impact on how efficiently it can be solved. For that reason, this assignment focuses not only on obtaining solutions, but also on understanding the importance of state-space design, valid actions, transition rules, and algorithm selection.

Hypothesis

We hypothesize that both problems can be solved effectively once they are represented with an appropriate discrete state space. For Problem 3.7, we expect that a visibility-graph representation together with an informed search method will produce an efficient shortest-path solution. For Problem 3.9, we expect that breadth-first search with repeated-state checking will find the optimal solution, since each boat crossing has the same cost. We also expect these problems to show that search performance depends not only on the algorithm itself, but also on how well the problem is formulated from the beginning.

Experimental Design Choice

The experimental design focuses on two search problems, each approached by selecting an appropriate state representation and then applying algorithms suited to the structure. In 3.7, the continuous plane was replaced with a finite visibility graph connecting polygon vertices, the start, and the goal. Which preserved the geometry while allowing standard graph search. The algorithms were compared, one being A* the most optimal and efficient, also UCS which was also optimal, but slower, and Greedy BFS which was fast, but suboptimal. For 3.9 each state was represented as missionaries, cannibals and boat position. To encapsulate everything necessary to validate the states,

determine actions, and find the goal. And, since all crossings cost equally, BFS was used with repeated state checking to ensure optimal 11 step solution without revisiting states.

Methodology

The experiment was structured to support a controlled and reproducible comparison between two optimal and one heuristic only, applied to find the shortest path through a polygonal obstacle environment. The implementation uses a modular architecture that separates the environment representation, state space, and search algorithms; This ensures all three agents operate over an identical graph.

Environment

The environment for problem 3.7 consists of a two dimensional plane, containing eight polygonal obstacles of varying shapes and sizes, with a fixed start position(S) and goal position(G) placed outside obstacles boundaries. Because the state space is continuous and infinite, a finite reduction is applied before any search starts. The shortest path must consist of straight line segments whose endpoints are vertices, the start or the goal. This allows the space to be collapsed into a set of 35 nodes, forming the basis of the visibility graph.

For 3.9, the environment is a river crossing scenario in which three missionaries and three cannibals must be transported from the left bank to the right back, using a boat that holds at most two people, to the constraint that missionaries must never be outnumbered on either bank. The state is represented as a triple (m, c, b), where m and c are the counts of missionaries and cannibals remaining on the left bank, and b indicates the current boat position. Applying the safety constraint to all possible triples yields 20 valid states, of which 16 are reachable from the initial configuration.

Visibility Graph

The visibility graph is constructed over the reduced state space by testing every pair of nodes for mutual visibility. Two nodes are considered mutually visible if the line segment connecting them does not pass through the interior of any obstacle. Interior penetration is verified by sampling the three spaced interior points along each edge. Edges that pass all tests are added with a cost equal to the distance between the two points, yielding a graph of 111 edges that serve as the search space.

Search Algorithm

For problem 3.7, three best first search variants were implemented over the visibility graph using a shared minheap: A* with function $f(n) = g(n) + h(n)$, where h is the Euclidean distance to the goal, Greedy with $f(n) = h(n)$ only; and UCS with $f(n) = g(n)$ only. A* and UCS both found the optimal path, while greedy found a suboptimal path but expanded only 3 nodes compared to A*'s 14 and UCS's 31.

For problem 3.9, Breadth First Search was applied over the finite state graph, which guarantees an optimal solution in terms of number of crossings since every edge has uniform cost. Unlike the 3.7 problem, this state graph contains cycles, so a visited set is mandatory to prevent infinite loops. BFS returned the optimal solution of 11 river crossings.

Results and Analysis

3.7:

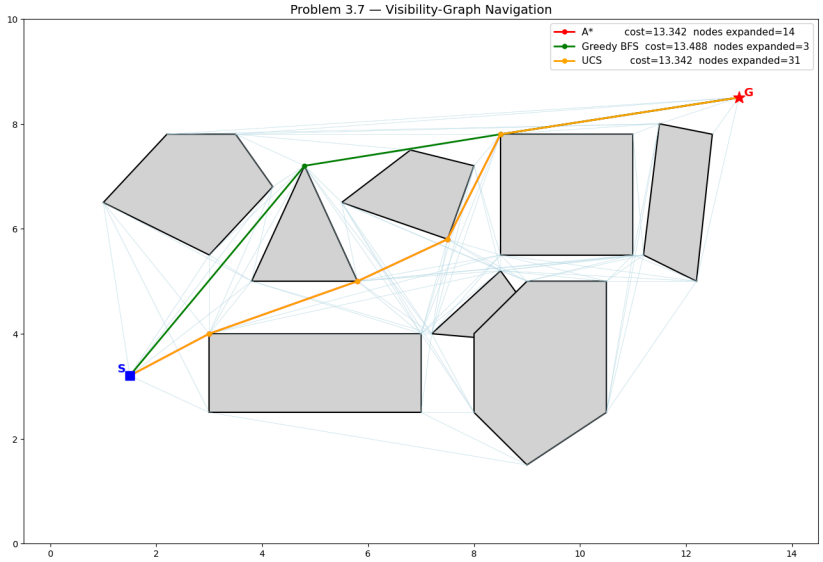

Fig.1. Graph 1 Problem 3.7

This exercise is focused on evaluating how different informed and uninformed search algorithms behave when navigating a continuous environment through a visibility graph. The goal is to determine how efficient each method can identify a path between two points while circumventing obstacles. For this, we measure two aspects, while using the same graph: the total cost of the final path, and the number of nodes of each algorithm.

Algorithm	Path Cost	Nodes expanded	Optimal?
A*	13.34	14	Yes
Greedy	13.49	3	No
UCS	13.34	31	Yes

Fig 02, Table 1, Algorithms search results

When comparing A* and Greedy, we can notice some differences in their results and how they search. Greedy expanded his search in a faster manner, travelling as directly as possible to the goal, taking far less nodes to reach the goal. While that focus makes it efficient, since it uses minimal computation, it also means that it ignores the cost accumulated along the way.This results in a faster yet less optimal path, since it compromises the accuracy in favor of speed.

A*,meanwhile, expands in more nodes because it considers the total cost so far and the heuristic estimate. This makes the process slower but more deliberate, evaluating multiple paths before deciding on the best course of action. The extra computation tends to pay off since A) it finds the shortest path and B) The cost is almost minimal and leads to optimal results. While Greedy favors speed, A* favors precision and accuracy.

Uniform Cost Search(UCS) serves mainly as a reference for validating the optimal route. As seen in figure 1, its results match those of A*, confirming that the heuristic used in A* is admissible and does not overestimate the true cost. However, since UCS lacks heuristic guidance, it tends to expand toward more nodes, making it less efficient even though it guarantees the optimal path.

Overall, the data presented shows that Greedy demonstrates a high efficiency, yet fails to guarantee optimal results. A* achieves optimality by exploring a broader portion of the search, while UCS confirms the correct path but with reduced efficiency.

3.9:

This exercise evaluates how an uninformed search algorithm behaves when solving a constrained river-crossing problem through a finite state space. The goal is to determine how efficiently Breadth-First Search can identify the optimal sequence of crossings while always satisfying the safety constraint that missionaries are never outnumbered by cannibals on either riverbank.

Value	Result
Total Valid states	20
Reachable	16
Optimal Crossings	11
Algorithms used	BFS
Optimal solution found	Yes

Since every river crossing has the same cost, BFS was selected because it guarantees the shallowest and therefore optimal solution under uniform step cost. Since there is no heuristic involved in this problem, BFS explores the state space level by level until reaching the goal.

Repeated-state checking is also essential here because the state space contains cycles. For example, sending one person across and then bringing one back can return the problem to a previously visited state. Without a visited set, the search would keep revisiting the same configurations and waste computation. By marking explored states, BFS expands each reachable state only once, which keeps the search correct and efficient within this small state space.

Conclusions

For problem 3.7, the goal was to reduce an intratable continuous plane, with infinite states, into a 35 node graph by demonstrating that optimal paths need only bend at polygon vertices. Running A*, Greedy BFS, and Uniform Cost Search(UCS) on the graph verified the theoretical differences. A* and UCS both presented 13.342 as the optimal path cost, while GBFS found 13.488 by ignoring accumulated cost. A*'s stood out by expanding only 14 nodes while UCS had 31. This demonstrated how a well chosen admissible heuristic dramatically reduces search effort without losing optimality.

In 3.9, a careful state space design compressed a complex puzzle into 20 valid states of which 16 were reachable, allowing BFS to find the 11 crossing optimal solution. The analysis showed why people usually fail this puzzle. It requires them to do counter intuitive backward moves and disciplined backtracking in which algorithmic search outshines the rest. This assignment demonstrated how choosing the right state representation and heuristic is usually more impactful than the chosen algorithm.

Lessons Learned

1. State space design is the most impactful decision.
2. Admissible heuristics deliver real computational gains.
3. Optimality comes at a cost, but so does disregarding it.
4. Repeated state checking is unavoidable for cyclic space
5. Formal problem formulation sharpens algorithmic thinking.
6. Thoughtful problem formulation is often more impactful than the choice of algorithm itself.

AI Disclaimer:

AI was used to make visible graphs in Jupyter Notebook, except the diagram of 3.9 a.

References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] aimacode, "Artificial Intelligence: A Modern Approach Code Repository." [Online]. Available: https://github.com/aimacode. [Accessed: Apr. 22, 2026].

Group Collaboration

1. Diego - Methodology, worked on the analysis for Problem 3.9
2. Malik - Methodology, worked on the analysis for Problem 3.7
3. Edgard- Introduction, experimental design 3.9 c
4. Marco - Conclusion and lesson learned. Also, 3.9 b.

Video Link: https://youtu.be/YnrRiycwK5Y